

Code review — Variante BAC

Runde de review pe variantele tale de bac, cea mai nouă sus.

Runda 4 — 2026-09-29 — sha 61e5cf0

Ai livrat 8d0a472 „final tema teste”, adus pe master prin merge (61e5cf0). Două variante noi scrise de la zero — 10 și 11 — plus exercițiul 4 din varianta 9.

Un lucru de lămurit înainte de orice: ai lucrat pe branch-ul feat , pornit din d1f02aa , adică **dinainte să primești runda 3**. Nu e o greșeală — e ordinea în care s-au întâmplat lucrurile. Dar înseamnă că **niciuna dintre cele cinci constatări 🚫 ale runde 3 nu e atinsă**, și că unele dintre ele se repetă în codul nou. Citește runda 3 înainte de a te apuca de asta.

Poarta de compilare: PICATĂ. Trei erori, toate în fișierele scrise azi. Deci „final tema teste” n-a trecut niciodată prin Build . Detaliile la B1.

Fișier	Runda 3	Acum
temaVar1.h	✓	✓ (neatins)
temaVar2.h	✓	✓ (neatins)
temaVar3.h	✗	✗ (neatins)
temaVar4.h	✓	✗ (neatins — vezi nota)
temaVar5.h	✗	✗ (neatins)
temaVar6.h	✓	✓ (neatins)
temaVar7.h	✓	✓ (neatins)
temaVar8.h	✓	✓ (neatins)
Var9.h → temaVar9.h	✓	✗ regresie
temaVar10.h	—	✗ nou
temaVar11.h	—	✗ nou
algortimi-utili.h	✗	✗ (neatins)

Nota la `temaVar4.h` : în runda 3 l-am dat ✓ pentru că, incluse toate fișierele împreună, apelul lui orfan `f(n,x+1)` se lega tăcut de altă funcție. Singur, n-a compilat niciodată. Vezi B4 — e exact capcana pe care ai adâncit-o acum.

Ce ai închis

Din runda 3	Stare
M4 — <code>Var9BAC.TXT</code> gol	✓ are acum datele din exemplu

Atât. Restul e mai jos.

Critice

B1 — proiectul nu compilează, și pică fix pe ce ai scris azi

`main.cpp:2-3` , `temaVar11.h:154` , `temaVar10.h:127` , `temaVar10.h:157`

CUM E ACUM. Compilat exact ca tine (`main.cpp` , `-Wall` , `gnu++11`):

```
temaVar11.h:154: error: use of undeclared identifier 'citVar10';
               did you mean 'citVar11'?
temaVar10.h:127: error: redefinition of 'rezVar10'
               temaVar11.h:152: note: previous definition is here
temaVar10.h:157: error: unknown type name 'Calcul'
```

Trei erori, toate în cele două fișiere livrate azi. În plus `main()` e gol (`main.cpp:8`): chiar dacă ar compila, n-ar rula nimic. În runda 3 `main()` apela `Var9sub(3,5)` — ai pierdut și asta.

DE CE. Compilatorul lucrează pe **o singură unitate de compilare**: ia `main.cpp` , lipește în el tot ce scrie în `#include` , în ordinea în care apar, și abia apoi verifică. Ordinea ta e `temaVar11.h` **întâi**, `temaVar10.h` al doilea. Când ajunge la linia 154 din `temaVar11.h` , `citVar10` încă nu există nicăieri — e definit într-un fișier care se lipește *mai târziu*. De-aia eroarea zice „undeclared”, nu „not found”: în C++ o funcție trebuie să fie **văzută înainte** de locul unde o chemi, nu doar să existe undeva.

A doua eroare vine din același mecanism: după lipire, `rezVar10` apare de două ori în același fișier. Gărzile `#ifndef` te apără de includerea aceluiași fișier de două ori — nu de două fișiere diferite care definesc același nume.

A treia (`Calcul final:` la `temaVar10.h:157`) e un rând de text rămas între două comentarii. Compilatorul îl citește ca declarație de variabilă de tip `Calcul` .

FIX. `citVar10` → `citVar11` · funcția `rezVar10` din `temaVar11.h` se numește `rezVar11` (vezi B3) · rândul 157 intră în comentariu. Apoi: **apasă Build înainte de commit**. Un commit care nu compilează nu e „final”, e o copie de siguranță.

B2 — ai pus în `NR.TXT` răspunsul, nu datele; și ai stricat varianta 2 cu el

`NR.TXT` , `temaVar11.h:127` , `temaVar2.h:124`

CUM E ACUM. Enunțul variantei 11 spune: fișierul conține pe a doua linie `4 6 3 7 8 1 6 2 7 9 10 8`, iar programul trebuie să afișeze `4 6 6 7 8 8 8 8 8 9 10 10`. În `NR.TXT` ai scris:

```
12
```

```
4 6 6 7 8 8 8 8 8 9 10 10
```

Adică **ieșirea așteptată**, pusă pe post de intrare.

Și mai e ceva. `NR.TXT` nu era liber: e fișierul variantei 2, care cerea „numerele naturale nenule, în ordine crescătoare”. Conținea `-3 -10 0 7 -5 7 51 -800 6 3798` — exact exemplul din enunțul ei. L-ai suprascris. Rulat `citire3` din `temaVar2.h` pe fișierul de azi:

```
NR.TXT vechi (varianta 2)    -> 7 7 51 6 3798
NR.TXT de acum (varianta 11)-> 12 4 6 6 7 8 8 8 8 8 9 10 10
asteptat de enuntul var. 2  -> 6 7 7 51 3798
```

DE CE. Algoritmul tău de la varianta 11 înlocuiește fiecare element cu maximum dintre el și vecinul din stânga. Pe un șir **deja crescător** nu are ce să schimbe — rezultatul iese identic cu intrarea. Deci testul tău trece **oricare ar fi codul din funcție**: ai fi putut scrie o funcție goală și tot ai fi văzut pe ecran șirul așteptat. Un test care nu poate să pice nu-ți spune nimic; îți dă doar senzația că merge.

Partea a doua e regula de igienă din spate: **numele fișierului de date face parte din cerință**. Varianta 2 cere `NR.TXT`, varianta 11 cere `numere.txt`. Ai trei cerințe care se bat pe același fișier — ți-am semnalat asta în runda 2, ca risc. Acum e pagubă: datele variantei 2 nu mai există în director (sunt doar în istoricul git).

FIX. `NR.TXT` primește înapoi datele variantei 2. Varianta 11 își face `numere.txt`, cu **intrarea** din enunț, nu cu ieșirea. Apoi rulezi și compari ce apare pe ecran cu ce scrie în enunț — asta e verificarea, nu faptul că programul n-a crăpat.

B3 — ai copiat fișierul 10 în 11 și ai redenumit jumătate

```
temaVar11.h:152-157
```

CUM E ACUM. În `temaVar11.h`, funcția care rezolvă varianta 11 se numește `rezVar10` și cheamă `citVar10`:

```
void rezVar10(){           // ar trebui rezVar11
    int n,v[30000];
    citVar10(n,v);         // ar trebui citVar11
    Var11ex3(v,n);
    afiVar11(v,n);
}
```

Celelalte trei nume din fișier sunt corecte: `citVar11` , `Var11ex3` , `afiVar11` . Deci ai redenumit trei din cinci.

DE CE. E **exact** tiparul de la B1 din runda 3, în care ai redenumit `f` → `Var3f` / `Varf` / `Var5f` dar ai lăsat apelurile recursive din corp. Acolo aveai o explicație: apelul era ascuns în interiorul funcției redenumite. Aici e altceva — ai luat fișierul 10 ca șablon (bună idee) și ai trecut prin el schimbând `10` în `11` **unde ți-a sărit în ochi**: la definiții, pentru că acolo numele e primul cuvânt de pe rând. La `void rezVar10(){` și la `citVar10(n,v);` numele nu e primul lucru pe care-l citești, deci n-a fost văzut.

Nu e neatenție întâmplătoare: e mereu **aceeași jumătate** care scapă — cea în care numele nu stă la începutul rândului. De-aia ți-o semnalez a doua rundă la rând.

FIX. Când copiezi un fișier ca model, nu-l citi ca să găsești numele: caută-le. `Ctrl+Shift+F` pe `Var10` în fișierul nou trebuie să întoarcă **zero** rezultate. Și `rezVar10` → `rezVar11` , altfel rămâi cu coliziunea de la B1.

B4 — `int f(int n)` din `temaVar11.h` e momeala care ascunde bug-ul din runda 3

`temaVar11.h:82` , `temaVar3.h:64` , `temaVar4.h:84` , `temaVar5.h:73`

CUM E ACUM. În runda 3 ți-am arătat că `Var3f` , `Varf` și `Var5f` cheamă în corpul lor un `f(...)` care nu mai există sub numele ăsta. Compilele singure: `error: use of undeclared identifier 'f'` . Compilele împreună cu celelalte fișiere: **zero erori**, pentru că se legau tăcut de `f` -urile rămase în `temaVar1.h` și `temaVar2.h` . Reprodus atunci: `Var3f(26)` → `808y` în loc de `xyyy` .

Azi ai scris în `temaVar11.h` o funcție globală numită exact `f` :

```
int f(int n){ ... }
```

DE CE. C++ alege funcția după **nume plus tipurile argumentelor**. `f(x-1)` din `Var3f` primește un `int` — și noua ta `int f(int n)` se potrivește perfect. Deci în clipa în care `main.cpp` va include și

`temaVar3.h`, apelul orfan nu va mai da nicio eroare: se va lega de funcția variantei 11 și va calcula cu totul altceva.

Ăsta e lucrul de reținut din runda asta: **o eroare de compilare e un cadou**. Bug-ul de la B1/runda 3 e periculos tocmai pentru că a încetat să fie eroare. Cu cât ai mai multe funcții numite `f` într-un proiect, cu atât mai multe greșeli devin tăcute.

FIX. Repară întâi B1 din runda 3 (apelurile din `Var3f` / `Varf` / `Var5f`), apoi redenumeste și `f` din `temaVar11.h` în `Var11f`. Regula, pentru tot proiectul: **niciun nume de funcție nu se repetă între variante**. Prefixul `VarNN` există fix ca să nu se poată.

B5 — varianta 9: exercițiul 4 început și abandonat, iar fișierul a căzut

`temaVar9.h:181`

CUM E ACUM. Ai redenumit `Var9.h` → `temaVar9.h` (bine, e convenția ta) și ai adăugat exercițiul 4. Dar:

```
if(x>=100)&&x<=999){
```

```
temaVar9.h:181: error: expected ';' after expression
```

Paranteza se închide după `x>=100`, deci `if` se termină acolo, iar `&&x<=999){` rămâne un rând fără sens. `Var9.h` era singurul fișier **curat** la runda 3; acum nu mai e.

Și funcția e neterminată: marchează numerele văzute în `v`, dar nu caută nimic, nu afișează nimic și n-o cheamă nimeni. Cerința are trei părți — marchează, caută de la 999 în jos primele două nemarcate, afișează (sau `0` dacă nu există două).

DE CE. `if` are nevoie de o singură pereche de paranteze care să cuprindă **toată** condiția: `if( ... )`. Tu ai scris `if(x>=100)` — condiție completă, gata — și apoi ai mai pus ceva după. Din felul cum arată, ai scris întâi `if(x>=100)` și ai adăugat a doua condiție fără să muți paranteza de închidere. Se întâmplă des; de-aia compilezi des.

Al doilea lucru: `v[x]=1` cu `x` până la 999 cere ca apelantul să dea un vector de **cel puțin 1000** de elemente. Nu se vede din semnătura `void Var9Citire(int v[])` — în C++ vectorul se transmite ca adresă, iar mărimea lui nu călătorește cu el. E pe răspunderea ta.

FIX. `if(x>=100 && x<=999)` . Apoi termină cerința: vector de 1000, parcurs de la 999 în jos, oprit după al doilea nemarcat.

B6 — două răspunsuri de teorie greșite

`temaVar11.h:14` , `temaVar10.h:30`

CUM E ACUM.

Varianta 11, exercițiul 1. `x=8` , `y=6` , se cere expresia cu **valoarea 0**. Ai răspuns `a` , cu justificarea `3*8-4*6 = 24-24 = 0` . Calculul e corect, răspunsul nu. Valorile reale:

Expresie	Valoare
a. <code>3*x-4*y==0</code>	<code>0==0</code> → 1
b. <code>(x+y)/2 > x*y+1</code>	<code>7 > 3</code> → 1
c. <code>!(x/2+2==y)</code>	<code>!(6==6)</code> → <code>!(1)</code> → 0
d. <code>x-y+3!=0</code>	<code>5!=0</code> → 1

Răspunsul e `c` .

Varianta 10, exercițiul 2. Ai scris `nr=2*10+2=42` , și nu ai notat rezultatul final. Rezultatul e **24**: `nr = 4 + 2*10` .

DE CE. La primul, diferența e între **expresia aritmetică** și **expresia din enunț**. `3*x-4*y` chiar dă 0 — dar expresia care se evaluează e `3*x-4*y==0` , iar `==` nu întoarce rezultatul scăderii, ci **adevărul comparației**: 1 dacă e adevărată, 0 dacă e falsă. Deci tocmai pentru că scăderea dă 0, expresia dă 1. Regula practică la grilele astea: operatorul care se evaluează ultimul decide valoarea — aici `==` , nu `-` .

La al doilea, ai calculat corect `[n/10]%10 = 2` și `p = 10` , dar ai scris cifrele în ordinea în care le-ai gândit (`2` și `10` → `42` ?) în loc să aduni: `4 + 20 = 24` . Tabelul tău se oprește fără rând de rezultat — la fel ca la `f(3)` din `temaVar8.h` , semnalat în runda 3. **Pune întotdeauna ultimul rând, cel cu ce se afișează.** Jumătate din greșelile tale de teorie apar în pasul pe care nu-l scrii.

FIX. `temaVar11.h:14` → `c` · `temaVar10.h:30` → `nr = 24` , cu rând de rezultat.

Restanțe din runda 3 — toate cinci, neatinse

Fișierele nu au fost deschise, deci constatările sunt neschimbate. Le repet condensat:

#	Unde	Ce
B1	<code>temaVar3.h:64</code> , <code>temaVar4.h:84</code> , <code>temaVar5.h:73</code>	apeluri recursive rămase pe <code>f</code>
B2	<code>temaVar3.h:63</code> , <code>temaVar5.h:73</code>	<code>'x'</code> și <code>'*'</code> cu apostrof tipografic (U+2019)
B3	<code>temaVar4.h:124</code>	enunțul cere diagonala secundară, codul face principala
B4	<code>temaVar5.h:166–170</code>	<code>construireVar5()</code> / <code>afisareVar5()</code> chemate fără argumente; <code>intj</code> la :158
B5	<code>temaVar9.h:57</code>	echivalența logică: ai <code>d</code> , corect e <code>b</code> (rulat: <code>b</code> diferă în 0 cazuri din 676, <code>d</code> în 336)

🟡 Importante

M1 — `main.cpp:2–3 : include 2 fișiere din 11`. A treia rundă la rând. Acum e și mai vizibil de ce contează: singurele două fișiere incluse sunt singurele în care ai văzut erori. Celelalte nouă n-au fost verificate de compilator de trei săptămâni.

M2 — `temaVar2.h:130 : citire3 nu sortează`. Cerința spune „în ordine crescătoare”. Rulat pe datele ei originale: `7 7 51 6 3798` , față de `6 7 7 51 3798` cât cere enunțul. Tu filtrezi pozitivele și le afișezi în ordinea din fișier. Lipsește sortarea.

M3 — restanțele 🟡 **din runda 3, neatinse:** `7****` are tot patru stele deși tabelul tău are cinci rânduri (`temaVar5.h:84` , rulat: `7*****`) · `Var2f` e tot goală (`temaVar2.h:197`) · `algortimi-utili.h` tot nu compilează (`v` și `n` redeclarate peste parametri, liniile 50–51).

Cleanup

C1 — `temaVar1.h:9` : răspunsul e tot `B` . Același exercițiu, rezolvat corect în `temaVar9.h:16` (`d`). A patra rundă în care apare.

C2 — `bin/` , `obj/` tot comise, tot fără `.gitignore` . A patra rundă.

C3 — commit-urile sunt tot semnate `KERNEL0S-PC\Administrator` .

C4 — `VARIANTE-BAC.layout` comis la fiecare commit. E fișierul în care Code::Blocks ține ce taburi aveai deschise. Intră în `.gitignore` împreună cu `bin/` și `obj/` .

Before / After — B2

	NR. TXT	Ce se întâmplă
Înainte	<code>-3 -10 0 7 -5 7 51 -800 6 3798</code>	datele variantei 2, din enunțul ei
Acum	<code>12 / 4 6 6 7 8 8 8 8 8 9 10 10</code>	ieșirea variantei 11; varianta 2 afișează gunoi
Corect	<code>NR.TXT</code> rămâne al variantei 2; varianta 11 primește <code>numere.txt</code> cu <code>12 / 4 6 3 7 8 1 6 2 7 9 10 8</code>	fiecare cerință cu fișierul ei, cu intrarea din enunț

```
// ÎNAINTE – citeste raspunsul si il "rezolva"
ifstream f("NR.TXT");          // contine 4 6 6 7 8 8 8 8 8 9 10 10

// DUPA – citeste intrarea, ca in enunt
ifstream f("numere.txt");      // contine 4 6 3 7 8 1 6 2 7 9 10 8
```

Ce ai făcut bine

Astea nu sunt politețuri — le-am verificat rulând codul tău.

Ce	Rezultat
<code>Var11ex3</code> — maximul prefixului, pe datele reale din enunț	rulat: <code>4 6 6 7 8 8 8 8 8 9 10 10</code> ✓
<code>Var10ex5</code> — pătratele perfecte pare, $n=2$ $p=3$	rulat: <code>0 4 16 / 36 64 100</code> ✓
<code>f(5)</code> și <code>f(23159)</code> — tabel pe 6 pași	<code>5</code> și <code>1</code> , corecte ✓
<code>f(3,1)</code> recursiv cu doi parametri	<code>9</code> , corect ✓
<code>9/2*2-5</code>	<code>3</code> , corect ✓
pseudocod $n=296385$, $k=3$	<code>15</code> , corect ✓
<code>abracadabra</code> fără <code>a</code>	<code>11 brcdbr</code> , corect ✓
<code>abbacdde</code> fără duplicate	<code>abacde</code> , corect ✓

Cei doi algoritmi pe care i-ai scris azi sunt corecți. Maximul prefixului e chiar soluția eficientă pe care o cere enunțul — o singură parcurgere, fără vector suplimentar. Și pătratele perfecte pare le-ai construit dintr-un contor care crește din 2 în 2, nu căutând pătrate: e exact ideea bună.

Diagnosticul rundei, într-o frază: gândirea e bună, cade la asamblare — nume copiate pe jumătate, fișiere care nu ajung la compilator, date de test care sunt chiar răspunsul.

Q&A — trei întrebări

- `temaVar11.h` conține `int f(int n)`, iar `temaVar3.h` conține `f(x-1)` în corpul lui `Var3f`. Azi, `main.cpp` nu le include pe amândouă. Dacă le-ai include, care dintre cele două lucruri s-ar întâmpla: apare o eroare de compilare, sau programul compilează și afișează altceva decât trebuie? De ce?
- Ai rulat varianta 11 și pe ecran a apărut exact șirul din enunț. Explică de ce asta **nu** dovedește că funcția e corectă. Ce ar fi trebuit să conțină fișierul ca rularea să însemne ceva?

3. La varianta 11 exercițiul 1 ai calculat $3*8-4*6 = 0$ și ai răspuns **a**. Calculul e corect, răspunsul nu. Care e, în cuvintele tale, diferența dintre valoarea lui $3*x-4*y$ și valoarea lui $3*x-4*y==0$?

Ce urmează

În ordinea asta — primele două durează un sfert de oră împreună:

1. **B1** — cele trei erori. Apasă **Build** până e curat.
 2. **B2** — **NR.TXT** înapoi la varianta 2, **numere.txt** nou pentru varianta 11, cu intrarea.
 3. **main.cpp include toate cele 11 fișiere**. O să apară erori — asta e scopul. Ele sunt restanțele runde 3, care te așteaptă din 23 septembrie.
 4. **Restanțele runde 3**, în ordinea din tabel. Începe cu B1: e aceeași greșeală pe care ai făcut-o azi la **rezVar10**.
 5. **B5 și B6** — varianta 9 terminată, cele două răspunsuri de teorie corectate.
-

Runda 3 — 2026-09-23 — sha d1f02aa

Prima ta reacție la un review pe repo-ul ăsta, în câteva ore. Ai umblat în șase din opt fișiere și ai scris varianta 9 de la zero. Fișiere stricate: de la 4 la 2. Cele cinci coliziuni de nume — toate închise.

main() apela efectiv ceva.

- **B1** — funcții redenumite (**f** → **Var3f / Varf / Var5f**), apelurile recursive din corp lăsate pe **f** · se legau tăcut de **f** -urile altor fișiere · **rulat: Var3f(26) → 808y** în loc de **xyyy** · **NEATINS** (runda 4, B1 restanțe — și repetat la **rezVar10**, runda 4 B3)
- **B2** — **temaVar3.h:63** bara **|** scoasă, **'x'** cu apostrof tipografic lăsat; **temaVar5.h:73** neatinsă · **NEATINS**
- **B3** — **temaVar4.h:124** enunțul reparat, codul lăsat pe diagonala principală · **NEATINS**
- **B4** — **temaVar5.h** semnături reparate, apelurile din **rezolvareVar5** rămase fără argumente; **intj=1** la :158 · **NEATINS**
- **B5** — **Var9.h:57** echivalență logică: răspuns **d**, corect **b** · **rulat pe toate perechile din [-5,20]: b diferă în 0 cazuri din 676, d în 336** · **NEATINS**
- **M1** **7****** cu patru stele (rulat: **7*******) · **M2** **main.cpp** include 2 din 9 · **M3** **Var2f** goală · **M4** **Var9BAC.TXT** gol · **M5** **algortimi-utili.h** nu compilează — M1, M3, M5 **NEATINSE**; **M2** **agravat** (2 din 11); **M4** **✓ ÎNCHIS**

-  **C1** `temaVar1.h:9` răspuns `B`, iar același exercițiu rezolvat corect în `Var9.h:16` · **NEATINS**, a patra rundă · **C4** `bin/` + `obj/` comise · **NEATINS**

✓ **Varianta 9 — 5 din 6 corecte, verificate prin rulare:** `Var9sub(3,5)` → `15 10 5` (prima funcție rulată de tine, din `main`) · `Var9f(1213111,1)` → `3` · `strcpy/strcat` → `abcd123efg` · pseudocod `n=103456→963` · pseudocod `n=1239→1` · citire `struct` → a. **B6 din runda 2 închis:** rulat pe `bacalaureat` → `baAcaAlaAuUreEaAt`.

Runda 2 — 2026-09-23 — sha 7796154

1137 de linii noi, variantele 5–8. Patru din zece fișiere nu compilau; `main()` era gol.

- ✓ **B2** — ghilimele tipografice + bara `|` · **închis în** `temaVar4.h`; în `temaVar3.h` doar pe jumătate și în `temaVar5.h` deloc (runda 3, B2)
- ✓ **B3** — `cif` definit de două ori în `temaVar3.h` · **închis** (redenumit `cifEx4`; a rămas enunțul dublat — runda 3, C5)
- ✓ **B4** — 5 coliziuni de nume între fișiere · **TOATE ÎNCHISE**, zero erori de redefinire (dar redenumirea a deschis B1 din runda 3)
-  **B5** — `temaVar4.h` rezolva cerința variantei 3 · **INVERSAT**: enunțul e corect acum, codul nu (runda 3, B3)
- ✓ **B6** — `cuvantNou` fără `'\0'` · **ÎNCHIS**, rulat curat
-  **B7** — `temaVar5.h`, variabile locale folosite ca globale · semnăturile reparate, apelurile nu (runda 3, B4)
-  **B8** — `f(3,17)` = `7*****` · mecanismul prins, numărătoarea tot greșită (runda 3, M1)
-  **M1** — `main()` gol · apelează ceva acum, dar include 2 fișiere din 9 (runda 3, M2)
-  **M2–M7, C1–C11** — `n` neinițializat în `rez3` · `citire3` afișează de două ori · `Var2f` fără return · `algortimi-utili.h` · `NR.TXT` partajat de 3 cerințe · litera de la `temaVar1.h:9` · `.gitignore` · identitatea git — **toate neatinsse**

 **Fișierul** `buletinAnalyze_2010.pdf` **din repo:** decizie luată, subiect închis.

Verificate prin rulare și corecte în runda 2: matricea pe coloane (V3), `3798` din `NR.TXT`, `Mare Frig Rosu` cu spații multiple, sumele parțiale `15/11/8/1`, `9+36+9=54`, prefixele `10305 1030 103 10 1`, cel mai mic divizor prim, pseudocodul cu `k` (22), `f(12345) = 42`.

Runda 1 — 2026-09-15 — sha 19f5193

Branch `master` , 4 fișiere, 500 de linii. `temaVar3.h` și `temaVar4.h` nu compilau.

-  **B1** — ghilimele tipografice + bara `|` din foaia de examen, în `temaVar3.h:63,67` și `temaVar4.h:82,87` · **NEATINS** (runda 2, B2)
-  **B2** — `rezolvare5` sărea consoanele și nu dubla vocala · **ÎNCHIS** (logica e corectă acum; a rămas terminatorul de șir — runda 2, B6)
-  **B3** — `varianta-1.h:10` , condiția gândită corect dar bifată litera `b` în loc de `d` · **NEATINS** (runda 2, C2)
-  **B4** — șapte cerințe fără rezolvare · **2 din 7 închise** (numerele divizibile cu n din `bac.txt`  rulat, și numerele cu cel mult 2 cifre din `NR.TXT` ); una rezolvată greșit (matricea cu diagonala secundară — runda 2, B5); patru rămân (runda 2, C1)
-  **M1** — `temaVar4.h` fără `#include <iostream>` · **NEATINS**
-  **M2** — `int A[100][100]` pentru `n,m ≤ 10` · **NEATINS** (runda 2, C5)
-  **M3** — `strlen()` recalculat în condiția buclei · **NEATINS**, acum în 3 locuri (runda 2, C6)
-  **M4** — `main.cpp` nu apelează nimic · **NEATINS**, acum `main()` e complet gol (runda 2, M1)
-  **M5** — `varianta-1.h` fără gardă de includere · **NEATINS** (fișierul se numește acum `temaVar1.h` — runda 2, C3)
-  **C1** — `minim` cu o ramură în plus · **NEATINS** (runda 2, C7)
-  **C2** — `bac1.txt` în loc de `bac.txt` · **NEATINS** (runda 2, C8)
-  **C3** — citește `m` înaintea lui `n` · **NEATINS** (runda 2, C8)
-  **C4** — `bin/` și `obj/` comise, fără `.gitignore` · **NEATINS** (runda 2, C9 — și cauza lui B1)

Verificate prin rulare și corecte în runda 1: `f(121,1) → 12345` , `f(12345) → 5310135` , `f(26)`

→ `xyyy` , matricea `min(i,j)` , mijlocul segmentului, `cif(a,b)` , și itemii cu variante de răspuns de la variantele 2, 3 și 4.

